

EDAN26 Lab instructions

Jonas Skeppstedt

August 24, 2026

No use of AI is permitted in the labs. In particular:

- It is *not permitted* to ask AI for a solution.
- It is *not permitted* to ask AI for finding a bug in your code.

But:

- It is permitted to ask AI explain *compiler messages* such as for syntax errors in your code.
- It is permitted to ask AI to explain concepts.
- It is permitted to ask AI how to install things and similar practical problems.

At least 90 % of the purpose of the labs is to have fun, develop your confidence, skills, experience, debugging methods suitable for yourself, self-insights and thinking process which are all harmed by AI in my opinion.

* C is required in Lab 2, and recommended in labs 3, 4 and 6. See Section 2.

1 How to do the labs

- To make lab presentation more efficient, you must have a Discord username that shows your real name (but to ask questions, you can use any username).
- Except for Lab 6, you can do the labs on Mac, Linux, or Windows with the Linux subsystem, and you can present them on any computer with a UNIX terminal and make, and least four cores, such as power8.cs.lth.se (Power).
- Hardware transactional memory only works on Power.

- Check out the sequential C program `preflow.c` in lab0 and the pseudo code in `preflow.pdf` in Tresorit and make sure you understand it. If not, ask at Discord or book a Zoom meeting with Jonas. Zoom is booked at <https://calendly.com/forsete>
- Then read the language and general requirements below and the specific requirements and questions for a lab. Each lab has hints at the end of this pdf which you might want to check out.

2 Language requirements

It is required to have parallelized preflow push in Erlang (lab 1), Java, C (lab 2), Rust (lab 5) and Clojure (lab 6). In labs 3, 4, and 6 you can use any language with support for automatic data-race detection (such as C and C++), and:

- Lab 3: barriers (or implement your own barrier)
- Lab 4: atomic variables
- Lab 6: hardware transactional memory

C is the most convenient choice.

Lab 5 has an assignment on strict functional programming (in Haskell) but with a simpler problem that is more suitable for strict functional programming.

3 Debugging and help from teachers

As a general rule, it is *not* expected that teachers will debug your code, partly because it can be too time-consuming and partly because it is an essential skill to develop. It is likely you will find C with the Google sanitizer plus printing lots of output the most efficient, plus using `gdb` if your program crashes (see the course book). Instead we are happy to look for "common mistakes" and give general hints on debugging parallel preflow push. If you need help to move forward, you can always book a Zoom meeting with Jonas (see above).

It can also be a good idea to write a question in a text-channel (lab or language specific) plus answering other questions! Feel free to suggest additional language-specific channels.

4 Lab requirements and questions

The teachers will ask you questions including these but it is not required to write the answers in the pdf.

General requirements

1. When you type the command `make` you should get output `PASS` and the execution time. Sometimes you need to type e.g. `make par` or modify the `makefile`.
2. All source code should be written by yourself or your lab partner, in which case you should understand it as if you had written it yourself.
3. No data-races or potential deadlocks are permitted, as reported by Google sanitizer or Helgrind (use smaller inputs for Helgrind).
4. You are allowed to use any library functions available in the language you use except if there is one for `maxflow`.
5. In the C/Pthreads labs, it is only in the lab on transactional memory that you may use non-standard extensions of ISO C, as shown in the hint.
6. You are not allowed to use timeouts to decide when your program “probably” has completed.
7. Sometimes it can be useful to select alternatives in the program based on values from the input, but that is not permitted, except that you may select what to do based on the number of nodes and edges. It is also permitted to determine general characteristics about the input such as how dense the graph is, or similar, but it is not allowed to check e.g. if there is an edge between nodes v_{124} and v_{129} and then select an alternative optimized for that input.

Lab 1: Erlang

Purposes

You will learn about:

- What actors are and how they communicate
- Guarantees about in which order messages arrive
- How messages that arrive out-of-order can cause your program to fail and how to solve that

Requirements

1. Each node must be one actor.
2. You are only allowed to share an edge object between two nodes but no other information (such as storing node heights somewhere easily accessible from any node or from a neighbor). You can use `erlang:ets` for the flows if you want, or you can use actors also for the edges.
3. If you use a controller, it is not permitted to let it periodically wake up nodes. Instead it should either keep track of which node is active, or solve it in some other way (as suggested by Nils Ceberg's solution).

Note

1. There is no performance requirement for Erlang.

Questions

1. If you use a controller, how can it know that no node is active? If you don't, how do the actors agree on terminating the algorithm?
2. How do you ensure messages to the controller do not arrive out-of-order and therefore could confuse e.g. a controller?
3. How many push messages can a node have in progress concurrently in your solution? (one is OK)
4. How do you ensure that a node never gets a negative excess preflow? (except possibly for the source depending on how you implement it)
5. Explain your source code.
6. Did you find a way to improve the performance of your program over the first correct version?
7. For which types of applications would you consider using Erlang and why?

Lab 2: Java and C with locks

Purposes

You will learn about:

- How to avoid deadlocks in multithreaded programs
- How to detect data-races in C automatically
- How to detect risks for deadlocks in C automatically
- The importance of a load-balancing
- The importance of a suitable task granularity that matches the structure of the program in order to reduce most of the overhead from thread synchronization. Recall that a task is the job performed as a “work-item” by a thread such as what is done with one node with excess preflow.

Requirements

1. Parallelize preflow-push both in Java and C with locks and possibly with condition variables (see hints about which language may be preferable to do first).
2. Check your C program both with the Google Thread Sanitizer (often called tsan) and Helgrind.
3. Print out a message when each thread terminates.
4. What fraction of the execution time is wasted on synchronization?
5. Collect and present statistics about load-balancing, such as for instance how many nodes each thread has processed.

Questions

1. How do you avoid data-races and deadlocks?
2. What is your approach to load-balancing and how does it succeed?
3. Is your parallel program faster than the sequential from Lab 0, and what do you think is the reason for this?
4. Did you notice any big performance difference between Java and C? (in principle, either can be fastest and the JVM and the optimizing compiler are important factors, and how you use the languages).
5. Explain your source codes.
6. Did you find a way to improve the performance of your programs over the first correct version?

Lab 3: Barriers

Purposes

You will learn about:

- How running threads in phases sometimes can reduce synchronization overhead and increase performance.
- How to use barriers.

Requirements

1. Either (1) modify your C program from Lab 2, or (2) use any other language of your choice that supports data-race detection, so that the threads run in two phases:
 - a first phase to let each thread decide on how much its nodes should push to a neighbor (or that a relabel is needed), and
 - a second phase with only one thread doing work to update the excess preflows and heights
2. Check your program for data-races as in Lab 2.
3. What fraction of the execution time is wasted on synchronization?
4. Collect and present statistics about load-balancing.
5. Your program should now be faster than the sequential from Lab 0.

Questions

1. Does the use of phases affect the possibility to do good load-balancing, and why, and if it does, to what extent is it significant?
2. Explain your source code.
3. Can you distribute the nodes in a fair way for load-balance?
4. Did you find a way to improve the performance of your program over the first correct version?

Lab 4: Atomic variables

Purposes

You will learn about

- The overhead of using atomic variables over non-atomic variables.
- When this overhead may save time from other parts of a program.

Requirements

1. Make a copy of the sequential C program from Lab 0 and change the type from `int` to `atomic_int` for each variable in the `node_t` and `edge_t` types., and start the file with `#include <stdatomic.h>`
2. Measure how this affects the execution time (without adding any threads).
3. Find an explanation for why or why not this affects the execution time by looking at the assembler code. See hint for this lab.
4. Modify your program from Lab 3 (or Lab 2 if the language used in Lab 3 has no support for atomic variables) so that it uses both barriers and atomic variables.
5. Your program should still be faster than the sequential from Lab 0.

Questions

1. What is your explanation to requirement 3?
2. With only one thread doing work in phase 2 in Lab 3, can you move some of that work to phase 1 instead?
3. Which memory order is used by default for compound assignment operators (such as `+=`) ? (in case you don't use C/C++, your answer may differ from these languages)
4. Will you get a data-race if you switch to `memory_order_relaxed` in C/C++? Why or why not?
5. Can you notice any performance difference when using (1) sequential consistency memory order and (2) relaxed memory order? Why or why not do you think?
6. Explain your source code.
7. Did you find a way to improve the performance of your program over the first correct version?

Lab 5: Rust and Haskell

Purposes

You will learn how Rust is used when

- different pointers need to point at an object: move and borrow
- multiple pointers/threads need to use an object: atomic reference counters
- moving objects when creating threads

You will also learn how a strict functional program can be parallelized in Haskell

Requirements

1. Make a sequential version of preflow-push in Rust
2. Make a parallel version of preflow-push in Rust
3. Make the sequential Haskell program to find and add prime numbers complete
4. Improve a strictly functional parallel version of this Haskell program

Note

1. There is no performance requirement for Rust or Haskell.
2. It is OK to use one shared set of active nodes. Optionally let each thread have its own set of active nodes.

Haskell

1. To install parallel Haskell: `sudo apt install libghc-parallel-dev`
2. Study the file `seq-sum.hs`
3. What does it compute?
4. Change it so it computes the sum of prime numbers.
5. In case you wonder, the purpose of `deepseq` is to force Haskell to compute the value of `s` instead of waiting until it is actually needed, i.e. in the `printf` that follows. This `deepseq` is required due to Haskell's *lazy evaluation* which normally makes measuring time confusing but makes it possible to have e.g. lists of infinite size.
6. Next study the file `par-sum.hs`

7. It first divides the work in two halves.
8. It then *lazily* computes the two halves in one thread but it does not actually do any work. It just notes the equations for how to compute each of `s1` and `s2`.
9. The work is forced to be done by the construct

```
par s1 (pseq s2 (s1 + s2))
```

in which `s1` and `s2` are computed in parallel and then used in `(s1 + s2)`
10. Make the same change to it so it computes the sum of prime numbers.
11. How many threads does it use?
12. Change it so it takes a parameter with the number of threads to use. It is likely a good idea to copy the function and make another one for this.

Optional

1. Make the parallel version faster than your program from Lab 4 but it is not at all certain you will succeed.

Questions

1. Explain Rust's rules about moving and borrowing objects.
2. Explain atomic reference counters.
3. Explain your source code for your two programs.
4. What do you think about parallelizing Rust code as compared with Erlang, Java and C?
5. When would you recommend using parallel Haskell?

Lab 6: TM, OpenMP, and parallelizing compilers

Purposes

You will learn about:

- hardware transactional memory in C
- software transactional memory in Clojure
- OpenMP
- parallelizing compilers

Requirements

1. Modify your program to use hardware transactional memory.
2. Make a sequential Clojure program for preflow-push.
3. Parallelize your Clojure program.
4. Parallelize matrix multiplication using OpenMP (copy mm.c to a new file and add -fopenmp) measure the performance. Make sure you have read the hint about accesses to the c-matrix.
5. Compare the performance of mm.c using the commands below. Time a.out after each compilation. You may want to press CTRL-C after 10 s if a.out has not yet finished.

```
clang -mcpu=power8 -O3 mm.c
gcc -fexpensive-optimizations -mcpu=power8 -O3 mm.c
gcc -ftree-parallelize-loops=80 -fexpensive-optimizations -mcpu=power8 -O3 mm.c
pgcc -tp=pwr8 -O4 mm.c -Mconcur=allcores
xlc -qarch=pwr8 -O5 -qsmp -qhot=level=2 mm.c
```

Note

1. There is no performance requirement for Clojure.
2. It is OK to use one shared set of active nodes. Optionally let each thread have its own set of active nodes.

Questions

1. Why is it more reasonable to use software transactional memory with Clojure than with C, C++ or Java?
2. How does Power detect conflicts between hardware transactions?

3. Why can you not use I/O in a transaction, and which instructions can be used on Power if you need to print something?
4. Explain your source codes.

5 Erlang

Lab input files

We assume you have copied the multicore directory from Tresorit. All labs use the same input files located in multicore/data. Each lab has its own check-solution.sh Bash script which will use these common input files. The first tests are from Lab 6 of EDAF05, except that we will not remove any edges (so c and p are ignored from that lab). There are also other tests. The graphs are undirected, have n nodes, numbered as $0..n-1$ with the source being 0 and the sink $n-1$.

Installing Erlang

You can use power8.cs.lth.se or install Erlang on your laptop for example. On Ubuntu/Debian, type:

```
sudo apt install erlang
```

and on macOS:

```
brew install erlang
```

Now give the commands:

```
cd multicore/lab1
make
```

This will compile and run a file preflow.erl in the lab 1 directory.

The program make looks for a file called makefile which contains commands to execute, and in this case, first compiler preflow.erl and if that succeeds, runs your program on a small graph, and then runs the check-solution.sh script:

```
erlc preflow.erl
erl -noshell -s preflow preflow -s init stop < i
sh check-solution.sh run.sh
```

The file run.sh is another Bash script with the same commands as above to run your Erlang program. The file i contains a small example graph:

```
3 2 0 0
0 1 10
1 2 2
```

The input format is as follows: the first line contains four values: n , m , c , and p where n is the number of nodes, m is the number of edges, and the ignored c and p . Then follow m lines, each with an edge. An edge is specified as u , v , and a

positive flow capacity. The edges are undirected and there can only be one edge between two specific nodes u and v in the input.

In this example, the source will first push 10 to node 1 which can push 2 to the sink, and then, eventually, push back 8 to the source, so the maximum flow is 2 and the program should print $f = 2$. However, since the Erlang program is incomplete, it will instead only print some debugging output.

You can disable the printing by changing `PRINT` to 0 on line 5.

Erlang

Actors in different languages are usually very similar. We will now describe them in general terms for Erlang. An actor is like a thread in that it can do things on its own, and an actor does one of two things:

- waits for a message to arrive, and
- processes the next message that it received.

Some important facts include:

- A message never interrupts an actor. The actor processes the current message and after that waits for new messages.
- To send a message to another actor, the sender does not wait for a reply.
- If an actor sends multiple messages to another actor, all these messages will arrive in the same order.
- There is no other order between messages.
- In general, it is possible to avoid data-races by regarding all received data as read-only, but for Erlang which is a functional language no shared objects are sent between actors. Let the graph use a table to store the current flows, and let each node actor keep its excess preflow and height as a secret never shared.
- How can you change the excess preflow or height of a node if variables cannot be modified??

The actors are permanent but each actor manages a node which is like fast fashion: Whenever you want to modify something, you instead create a new node with the updated data and let the actor forget the old node and instead continue with the new. The actor runs the function `node_loop` and when a node is modified, this function is called with the new node (and the old node is garbage collected).

- Using recursive calls will normally use lots of memory but if nothing happens in your code *after* (on the source code lines "below"), then Erlang will simply just jump to the start of `node_loop` and reuse the memory it has so the stack will not grow.

To send a message, a reference to actor is needed plus the message. For example:

```
Actor ! { self(), hello }
```

Here one actor sends a message to another called `Actor` and the message contains a reference to the sender and the so called term `hello`. A term is just an identifier.

In `node_loop`, the node actor can receive a `hello` message, reply to the sender and call `node_loop` to continue. Note that nothing happens (is executed) after the recursive call.

Creating an actor

An actor is created with the function `spawn` which needs the module name from the first line, a function for the actor to run, and a list of parameters.

6 Hints

Lab 1 hints

In the lab you need to extend the file `preflow.erl` so that it can find the maxflow. How can that be done? A start is:

- Push from the source node to each of its neighbors by sending a suitable message (which does not exceed the remaining flow capacity of the edge, see Lab 0).
- As long as a node, other than the sink, has an excess preflow greater than zero, it should push if it can, and if not, it should relabel.
- Somebody has to figure out when the algorithm has terminated. Let both source and sink send messages to a controller actor when their excess has changed. When they match no other node can have any excess preflow.
- If u tries to send preflow to v , it should fail if u is not higher than v . Note that after the moment u sends a message to v , the height of v may have increased.

When u wants to push to v , then u can send its height in the message and then v can decide if the push should succeed or not and tell u in a reply.

Lab 2 hints

- Even if you have never programmed in C before, it may be easier to get a correct program in C than in Java thanks to the Google thread analyzer (tsan) and Helgrind.

One year a group of two very good students made a quite complex Java solution which had a bug. Instead of helping them, I suggested they should re-implement the bug for the next week's lab on C. They did and could solve the problem almost immediately using Helgrind (and immediately fix the Java program as well).

- Always start with the simplest possible solution.
- Identify which data two threads may access concurrently such that at least one thread may modify it.

Try to identify when deadlocks can occur. For instance, if there is an edge (u, v) , and one thread t_0 is pushing from u and another t_1 from v , then if t_0 locks u before locking v , and t_1 locks v before locking u , there is a deadlock.

This situation is detected by tsan and Helgrind. In fact they detect that the threads try to take the locks in the opposite order and therefore warn that there is a risk for deadlock (even if one never happens during a particular execution).

Two techniques to avoid deadlocks are

- Impose a lock-order which for instance says that for an edge (u, v) , if a thread wants to lock both of u and v , it must lock them in a predefined order (for instance, u first if $u < v$).
 - Use try-lock instead of lock. If the lock was already taken, the thread is not blocked. If a thread is pushing from u and finds v locked, it can skip v and check the next edge instead — but see the next hint about doing a relabel too early.
- It is a bug to relabel a node if it actually could do a push.
 - If you for instance (1) have a shared set of nodes with excess preflow, (2) that set is locked when a thread t wants to remove from it (or add a node to it), and (3) t has nothing else to do while waiting, then a condition-variable can be used to wake up t when t should make a new attempt to do something with that set. While this may sound reasonable it will probably lead to bad performance due to contention to the shared set. Can you avoid using synchronization when a thread should take its next node to process?

Lab 3 hints

- Instead of locking nodes and edges as needed, preflow-push can be divided into two phases such that the heights and edge flows are not modified immediately. We can then have multiple iterations where each iteration consists of two phases.
- In the first phase all threads are allowed to read any node's height, and no relabel is permitted.
- If a relabel is needed, it can be performed in the second phase.
- How many times can the flow of an edge be changed per iteration?
- When should the excess preflow of a node be modified and when should the node be put in some thread's set of active nodes? Recall from Lab 1 on Erlang, we say a node is active if it has excess preflow > 0 .
- We have edge flow modifications, and for a node modifications of heights, excess preflows, and the next pointer (to the next active node in some thread's set). How can these "commands" be represented and who should perform them? One option is to let each thread use an array of structs so that each struct represents the modifications of one node.
- If you want have an array of structs but you don't know how many elements will be required, you can do essentially as follows:

```
typedef struct {
    int    x;
    int    y;
} something_t;

something_t*  a;      // pointer to the first element
int          c;      // storage capacity (number of elements)
int          i;      // next available element

c = 2;
a = malloc(c * sizeof a[0]); // memory for two elements
if (a == NULL)
    error("no memory");      // error is not a standard function
i = 0;

if (i == c) {
    something_t*  b;

    c *= 2; // double the capacity
```



```

        b = realloc(a, c * sizeof(a[0]));
        if (b == NULL)
            error("no memory");
        a = b;
    }

    a[i].x = x;
    a[i].y = y;
    i += 1;

```

When the array is no longer needed, you call `free(a)`, but you should not free the old array after using `realloc`, since that is taken care of by `realloc` if needed (it can happen that `realloc` found available memory after the end of `a` so no free was needed).

- Pthread barriers are useful to make sure all threads agree on in which phase work should be done.
- When a barrier is initialized, one parameter is the number of threads T that will use it. Each thread calls the `pthread_barrier_wait` function with the same barrier as an argument. At a call, the first $T - 1$ threads are blocked, and when the last thread has made the call, all threads are allowed to resume execution.
- Note that a memory barrier (as in the Linux kernel or Power instruction) is different from a Pthreads barrier.

Lab 4 hints

- To quickly find the machine instructions of some code, you can add a global volatile variable and do something with it that is easy to search for in the assembler file. Then compile with `-S` and look in the file with suffix `s`.

```

volatile int hello;
void relabel(graph_t* g, node_t* u)
{
    hello &= 0x1234;
    u->h += 1;
    hello &= 0x5678;

    enter_excess(g, u);
}

```

Then search for `0x1234` and `0x5678`, and you should see the assembler code for `u->h += 1` in between (the Power instruction for `&=` is `andi.` which

means and-immediate and (the meaning of the dot suffix) set condition-codes).

Lab 5 hints

- If you get an error message from cargo about an operation not permitted, it may help to remove the target directory (which will be re-created automatically when compiling the next time). So if you see:

```
Operation not permitted (os error 1)
```

then do

```
rm -rf target
```

but beware that `rm -rf` is a dangerous command that recursively removes the argument files without asking so be careful.

- Start with studying the `swish.rs` program which simulates a number of swish transactions. You can compile it with

```
rustc swish.rs
```

and run it with

```
./a.out
```

- Then make the sequential preflow-push program complete. It is in the directory:

```
lab5/seq-rust/src/main.rs
```

Type:

```
cd lab5/seq-rust/src
cargo run
```

which should compile and run an incomplete program.

- Suppose you have a lock for the source node, and want to set its height to n , and do

```
let mut source = node_array[s].lock().unwrap();
source.h = n as i32;
```

```
// continue with initial pushes
// and then run the while-loop
```

A problem with this code is that the lock taken for `source` is not unlocked until `source` goes out of scope, so if you continue with the rest of the algorithm and still have it locked, you cannot lock it again. Depending on your source code, you may need to unlock by putting the above code in a block:

```
{
    let mut source = node_array[s].lock().unwrap();
    source.h = n as i32;
}
```

The lock is unlocked when `source` goes out of scope at the `}`.

- Then copy the `seq-rust` directory

```
cp -r seq-rust par-rust
cd par-rust
cargo run
```

and parallelize it.

Lab 6 hints

- It is assumed you use a barrier and two phases. Most of the work is done in phase 1 but some of the work done in phase 2 can be moved to phase 1, as with atomic variables. The question is if we can avoid overhead due to locks or atomic variables, or improve load-balance, by using transactional memory.
- Note that with transactional memory we can easily use any normal syntax and not only be limited to what can be expressed with atomic variables. Updating of pointers, for instance, is trivial with transactional memory.
- You should start threads as usual and the difference is how the transactions can be used instead of other synchronization primitives.
- You need to identify where data-races can occur without any use of synchronization and then put that code in a block:

```
__transaction_atomic {
    /* in principle any amount of reads and writes */
}
```

- Too many memory accesses in transactions usually lead to conflicts and transaction-restarts.
- You need to compile with

```
gcc file.c -fgnu-tm -O3
```

- Too many memory accesses in transactions usually lead to conflicts and transaction-restarts.
- There is a good chance your program will work at the first attempt.
- Start with studying the `swish.clj` program. The purpose is to see how `ref` variables to array elements can be modified, and how incrementing or decrementing an attribute of an object can be done using `update`.
- Note `recur` used to call the same function recursively, and `first` and `rest` to get the data in a list and the rest of the list, respectively.
- Then make a sequential `preflow.clj` complete.
- Add threads using the same technique as in the `swish` program. As when using TM for C, your program should work at once.
- All C compilers on Power support the `--help` option but you probably don't want to read all. Some useful flags include:
 - `pgcc -Minfo=all`
 - `xlc -qreport` which will produce a file with suffix `lst`.
- Note that in the `matmul` function, the `c[k][j]` reference will result in very bad cache performance since each iteration of the inner loop is to a different row, which typically is not in the cache. Can you change that in a trivial way (which clang appears to miss)?